AGABI Backend Phase 2 — Final Architecture Baseline

Agabi Engineering

July 22, 2026

CONTENTS

AGABI Backend Phase 2 — Final Architecture Baseline

Status: FROZEN on approval. Supersedes RFC-1 and RFC-2 in full. **Scope:** the permanent knowledge foundation. Every future backend engine builds on this and does not renegotiate it.

---

0. Architecture Audit

RFC-1 and RFC-2 are inputs, not scripture. Seven contradictions were found between them. Two would have blocked implementation. All are resolved below and the resolution is binding.

0.1 Contradictions found and resolved

#	Contradiction	RFC-1	RFC-2	Resolution
C1	Entity naming	`Statement`	`Proposition` in schema, `Statement` in prose	**`Statement` everywhere.** RFC-2 shipped a schema and a narrative using different names for the same table — an implementer would have created both. Binding: the table is `Statement`; the word "proposition" is never used.
C2	`PART_OF` relationship	acyclic edge type alongside `REQUIRES`	**dropped entirely**	**Restored as a third graph.** Compositional containment (light reaction PART_OF photosynthesis) is neither a prerequisite nor a reinforcement. It is structural, acyclic, and needed for aggregation and topic rollup. RFC-2 lost it in the split. See §11.3.
C3	Assessment scheduling	built in Phase 2 (§19), per explicit decision	schema present, **absent from the roadmap**	**Restored to the roadmap** at 2E. The decision to author assessment alongside knowledge review stands — the reviewer already has the source open, which roughly halves the cost.
C4	Contradiction detection	defined only for SPO (`same subject, same predicate, overlapping context, different object`)	statements have seven `form` s	**Per-form detection.** SPO detection is one of seven rules. Non-SPO forms (conditional, quantified, causal, comparative, probabilistic, definitional) each need their own conflict rule or are explicitly marked undetectable. §14.4.
C5	Context identity	seven columns + unique constraint	`dimensions Json` , id = hash	**Canonical hashing specified.** A JSON column loses the uniqueness guarantee unless hashing is deterministic. Binding: keys sorted lexicographically, values normalised per dimension type, `id`

#	Contradiction	RFC-1	RFC-2	Resolution
				`= sha256(canonicalJSON)`. Without this, two identical contexts get two rows and matching silently fragments. §13.2.
C6	Research connectors	fully specified (§37)	**dropped**	**Restored.** §24.
C7	Observability integration	health providers, metrics, scheduled jobs (§45)	**dropped**	**Restored.** §28.

0.2 Weaknesses found in both, fixed here

Weakness	Fix
Multi-tenancy asserted via a `scope` column; never specified	§23 — full isolation model, enforced in the store, conformance-tested
Deferred decisions scattered as 🔬 markers	§34 — one enumerated register with owners and triggers
No stated architectural review criteria	§35 — the review this document must pass to be frozen
Trust ladder promotion rules described in prose	§26 — promotion and demotion as a deterministic state function
No specified behaviour when the two graphs disagree	§11.5 — a concept cannot be both `REQUIRES` and `REINFORCES` in the same direction; the DAG wins and the reinforcement edge is rejected

0.3 What carries forward unchanged

Curriculum as a mapping layer · opaque immutable identity with mutable slugs · append-only versioning · mandatory provenance with machine-checked grounding · knowledge types as a registry · no difficulty column · Postgres behind `KnowledgeStore` · the Phase-1 advisor walls.

---

1. Vision

Agabi should know things, and teaching should be the act of selecting what it knows for a particular learner — not the act of asking a language model to invent a curriculum.

Today it knows nothing. `defaultOutline(topic)` is string templating; its entire knowledge of photosynthesis is the word “photosynthesis”. Everything a student learns comes from the weights of a free-tier model selected at request time — unversioned, unattributed, unverifiable.

Phase 1 secured the machinery and left the cargo unguarded. A model cannot advance a cursor. It can teach a fifteen-year-old that respiration reverses photosynthesis, and the architecture will faithfully deliver it.

Phase 2 closes that with the same invariant, extended one layer down.

1.1 The perfect outcome

Twenty years from now a maintainer can:

- ask what Agabi knows about consideration in contract law and receive statements scoped by jurisdiction, each traced to an authority, each with a validity window and a trust level
- replay a lesson from 2026 exactly, including which statement versions and which graph release were in force
- add a new board by inserting mapping rows and creating zero concepts
- add a new knowledge type, asset type, or context dimension by adding a registry row and touching zero tables
- discover that a concept was two concepts all along, split it, and have every historical reference resolve honestly rather than wrongly
- swap the storage engine by changing one module

If any of those requires a migration, the corresponding section is wrong.

---

2. Philosophy

P1 · Knowledge is not documents. A book is one serialisation, optimised for linear reading, frozen at an edition. Books, syllabi, papers and websites are *sources*. None is the architecture. The curriculum hierarchy must never be a parent of knowledge.

P2 · Truth is contextual. *"A contract requires consideration"* is true in India, false in France. *"First-line treatment is X"* was true in 2018. *"F = ma"* is true Newtonian, false relativistic. *"The octave has twelve semitones"* is true in Western tuning, false in Hindustani classical. A concept is a stable identity; a statement is a contextual assertion. Truth is not globally unique, and pretending otherwise forces a rewrite.

P3 · The model is an advisor. Extended from Phase 1: no model may establish knowledge. It proposes; the platform decides; humans establish trust. Enforced by the type system and the import graph, not by convention.

P4 · Evidence over conclusions. Store what cannot be recomputed. Ratios, scores, difficulty, mastery and recommendations are derived at read time, so improving a model reinterprets history instead of invalidating it.

P5 · Nothing is destroyed. Corrections are versions. Retractions are marks. Merges leave tombstones. The single exception is erasure of personal data, which is why learner observations live in a separate store.

P6 · Knowing is not teaching. *Photosynthesis converts light to chemical energy* is in every textbook and every model. *A fifteen-year-old who just heard "plants make food from sunlight" will assume the plant eats the sunlight, and the equation must not land on top of that* is in neither. The second is the product.

P7 · The platform works empty. Every component has meaningful behaviour on zero rows. Search returns nothing; traversal returns nothing; teaching falls back and records a miss. Coverage is incremental by construction.

---

3. First Principles

3.1 What teaching irreducibly requires

Q	Question	Implies
Q1	What is being learned?	stable identity → **Concept**
Q2	What is true of it?	assertion, separable from identity → **Statement**
Q3	Under what conditions?	**Context**
Q4	What must come first?	**Dependency graph**, acyclic
Q5	How do we know?	**Provenance + trust**
Q6	How is it best taught?	**Teaching asset**
Q7	Did the learner get it?	**Observation**

Absent from that list: subject, chapter, class, board, page, difficulty. Every one is organisational convenience, not a requirement of teaching.

3.2 Subject is not universal

Is *rate of change* Mathematics or Physics? Is *osmosis* Biology or Chemistry? Every answer is defensible, so the question is malformed. Subject is how an institution divides teaching labour; it varies by board, country and decade.

Binding: `subject` is a tag, many-valued and editable. It never appears in identity, including inside identifiers.

3.3 Chapters are pagination

NCERT renumbered chapters between editions. The knowledge did not change. **Binding:** no concept has a chapter. Curriculum is a mapping.

3.4 Textbooks are not authoritative about reality

NCERT is authoritative *for CBSE examinations* — a real and useful authority. It is not authoritative about physical reality. Curricular authority and epistemic authority usually agree; when they disagree both must be representable, and an exam-preparing student must get the curricular answer while the platform retains the knowledge that reality differs.

Binding: `authority` is a first-class field on a statement.

3.5 The smallest unit

Concepts are **entities**; statements are **assertions about them**. *Chlorophyll*, *Light energy* and *Photosynthesis* are concepts; *"chlorophyll absorbs light energy"* is a statement linking two of them.

The argument is decisive: **identity must be stable under changing belief.** Everything known about chlorophyll may be revised; chlorophyll remains the same referent. A model where a corrected fact creates a new entity accumulates orphans, not knowledge.

3.6 Which constraints are real

Claim	Real?	Why
Knowledge dependency is directional and acyclic	**yes**	if a genuine cycle existed in dependency, no learner could ever begin, and the subject would be unlearnable
Learning is iterative and cyclic	**yes**	optimisation deepens functions; respiration illuminates photosynthesis
Truth can be context-dependent	**yes**	logical
Forgetting occurs	**yes**	physical
Verification capacity is finite	**yes**	**the binding constraint of this phase**
Subjects	no	convention
Chapters	no	pagination
Difficulty is a property of content	no	it is a relation between content and learner
A lesson covers one topic	no	inherited from books

The first two are the reason there are two graphs, not one.

3.7 The bottleneck

Not review speed. **The ratio of human attention to justified knowledge.** Doubling review speed halves a number that remains 10,000 person-years.

Therefore the architecture attacks the denominator: deterministic validators, cross-source corroboration, contradiction detection against already-verified knowledge, and inference. The decisive property is that **contradiction detection makes human effort per statement fall as the graph grows** — the only mechanism that makes 100M concepts arithmetically conceivable.

---

4. Mental Models Applied

Recorded as findings, not as method.

Inversion produced §32's premortem, which produced the labelling invariant, the split operation, and the module-level import rule keeping the two graphs apart.

Falsification produced §35's review criteria and the prediction in the roadmap that grounding alone yields little and teaching assets yield much — stated in advance so it can be wrong.

Red-teaming killed four RFC-1 claims and, on a second pass, killed two of the kills. Both reinstatements were cases where the attack correctly found that something was wrong and incorrectly identified what.

Bottleneck targeting moved the design's centre of gravity from the schema to review throughput, and then from review throughput to the *ratio*, which is where it belongs.

Reverse engineering from "the world's best learning platform" produced §13 — because the thing that must be true underneath is that the platform knows *how to teach*, not merely what is true.

---

5. Success Conditions

#	Condition	Verification
S1	Knowledge is independent of curriculum	drop all program rows; graph unchanged and teachable
S2	Contextual truth without duplicate identity	two contradictory statements, one concept, both trusted, different contexts
S3	Trust is graded and never silent	nothing below the policy floor is returned unlabelled
S4	Every trusted statement traces to a source	quote literally present in its chunk
S5	Nothing is destroyed	no delete path outside learner erasure
S6	Any lesson is reconstructable	statement versions + release + assets recovered by lesson id
S7	New types need no migration	registry insert only
S8	New curricula need no new concepts	second board maps entirely onto existing concepts
S9	Storage is replaceable	second `KnowledgeStore` passes the same conformance suite
S10	The platform works empty	full suite green on zero rows
S11	Identity is permanent and meaningless	no FK targets `slug`; `Concept.id` never updated
S12	Dependency is acyclic; reinforcement is not	both tested, and the reinforcement test asserts cycles **pass**
S13	Determinism	identical bytes produce identical chunk ids
S14	Grounding is measurable	grounded / asset-supported / ungrounded distinguishable per lesson
S15	Tenancy is isolated	no query returns another tenant’s knowledge

6. Architecture Goals

Correctness before availability. Determinism before cleverness. Auditability before performance. Extensibility before completeness. **Every one of these is a tiebreak rule, applied when a decision is otherwise balanced.**

---

7. Non-Goals

Not built	Why	When
Mastery estimation	needs response data that does not exist	Phase 3
Digital twin, recommendation	depend on mastery	Phase 3+
Teaching Engine	Phase 2 builds its substrate only	Phase 3
Semantic/vector search	deterministic rungs suffice at current volumes; designed behind the same interface	when rung 3 measurably fails
Public HTTP knowledge API	no external consumer; the review UI drives route design	when one exists
Multi-language content	`language` dimension ships; translation workflow does not	post-CBSE
Automated curriculum alignment	would encode a guess before many manual mappings are observed	post-2F
Non-text source parsing	connector interface accepts them; no parser built	when a domain requires it
Graph database deployment	interface ready; implementation not needed	when measured

7.1 Anti-goals — forbidden, not deferred

- **Auto-promotion above `AUTO_VALIDATED`.** Under any backlog pressure, at any confidence.
- **Deletion of knowledge** in normal operation.
- **A difficulty column** anywhere.
- **A `verified: boolean`.** Trust has levels.
- **A curriculum foreign key on knowledge.**
- **Meaning inside identifiers.**
- **Serving verbatim source text to learners.**
- **Joining the knowledge and observation stores.**
- **A single unified edge table.**

Each is enforced by a test named in §29. If a test is deleted, the principle has been abandoned regardless of what this document says.

Part II — architecture and data model.

Part II — Architecture

8. Overall Platform Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│ L6  OBSERVATION      SEPARATE DATABASE. append-only. erasable.   │
│                      Mastery and efficacy are QUERIES over this. │
├──────────────────────────────────────────────────────────────────┤
│ L5  PROGRAM          Learning Programs → mappings onto L2.       │
│                      Delete all of L5 → L1–L4 unharmed.          │
├──────────────────────────────────────────────────────────────────┤
│ L4  TEACHING         how to teach it. misconceptions, analogies, │
│                      worked examples, diagram specs, Socratic paths. │
├──────────────────────────────────────────────────────────────────┤
│ L3  ASSERTION        what is true, under which conditions,       │
│                      at which trust level, on whose authority.   │
├──────────────────────────────────────────────────────────────────┤
│ L2  ENTITY           what exists. THREE graphs:                  │
│                      DEPENDENCY (acyclic) · COMPOSITION (acyclic)│
│                      · REINFORCEMENT (cyclic)                    │
├──────────────────────────────────────────────────────────────────┤
│ L1  SOURCE           documents, provenance, licence, checksum.   │
└──────────────────────────────────────────────────────────────────┘

References point DOWNWARD only. L2 references nothing above it.
L3 and L4 are PEERS. Neither derives from the other.
```

8.1 Relationship to Phase 1

Phase 1's three walls are extended, not replaced. `architecture.test.ts` walks `src/server` and forbids any file outside `advisors/` from importing an AI SDK — **with no `import type` exemption**. `Advice<T>` carries `unknown` and its only exit is `accept(advice, schema)`.

Knowledge extraction is the identical problem shape as intent classification: an untrusted model proposing something deterministic code must validate. **The extractor therefore lives under `advisors/` by force of the existing rule, returns `Advice<T>`, and cannot express a trust level above `MACHINE_PROPOSED` because its return type has no such field.** No new trust boundary is invented.

8.2 Integration point

Exactly one call site changes:

```
// manager.ts — startLesson()
const hits = await knowledge.resolve({ text: topic, context: ctx.learnerContext,
trustPolicy: policy });
const plan = hits.length ? await knowledge.selectPath(hits, ctx.learner, "LEARN",
policy, BUDGET) : null;
const proposed = plan ? outlineFrom(plan, topic) : defaultOutline(topic);
const { outline } = repairOutline(proposed, topic);
if (!plan) void emit(userId, EVENTS.knowledgeMiss, { topic }, "server",
sessionId);
```

`repairOutline`, `buildSkeleton`, `coerceSlot`, `adaptBlock`, `fillChunk` and every renderer are untouched. The visual guarantee, skeleton-first rendering and the fill ladder all keep working. **A knowledge platform that requires rewriting the teaching engine has failed its integration test before it is built.**

---

9. Component Architecture

```
src/server/
  knowledge/                    PLATFORM. pure + store. never imports advisors/
    concept.ts                  entities, aliases, tags, merge, SPLIT
    statement.ts                assertions; SPO is one form of seven
    context/ registry.ts match.ts canonical.ts
    graph/
      dependency.ts             REQUIRES. acyclic. topological sort.
      composition.ts            PART_OF. acyclic. rollup.
      reinforcement.ts          everything else. cycles legal.
      traverse.ts               bounded closure, shared
    trust/
      ladder.ts validators/ corroboration.ts contradiction.ts
      inference.ts policy.ts promote.ts
    teaching/ asset.ts registry.ts select.ts
    assessment/ item.ts registry.ts
    curriculum.ts search.ts version.ts review.ts path.ts ids.ts
    store/ KnowledgeStore.ts postgres.ts conformance.test.ts
  observation/                  SEPARATE STORE
    record.ts mastery.ts efficacy.ts earn.ts
  ingest/                       pure. no models, no store.
    parse/ clean.ts normalise.ts chunk.ts pipeline.ts
  advisors/knowledge/           UNTRUSTED. the only models.
    extractEntities.ts extractStatements.ts extractDependencies.ts extractAsset-
s.ts
  review/ queue.ts batch.ts decide.ts merge.ts split.ts
```

9.1 Wall rules (extending `architecture.test.ts`)

#	Rule	Prevents
W1	only `advisors/**` imports an AI SDK	Phase 1 wall
W2	`knowledge/**` never imports `advisors/**`	knowledge depending on proposal
W3	`ingest/**` never imports the store	pipeline writing directly
W4	`observation/**` never imports `knowledge/store`	store coupling
W5	nothing outside `store/**` imports Prisma	engine lock-in
W6	`graph/dependency.ts` never imports `graph/reinforcement.ts`	the two graphs silently re-merging
W7	`graph/composition.ts` never imports either of the others	same

W6 and W7 are the structural defence against premortem cause 5. A refactor that unifies the graphs must delete a test to succeed, which makes the decision visible.

---

10. Complete Data Model

Consolidated and binding. Naming resolved per C1: the entity is `Statement`*.*

```
// ═══════ L1 SOURCE ═══════
model Source {
  id String @id  kind String  title String  publisher String  authority String
  edition String?  publishedAt DateTime?  uri String?
  checksum String @unique  license String  licenseUrl String?
  ingestedAt DateTime @default(now())
}
model SourceChunk {
  id String @id                    // sha256(sourceId + locator + normalisedText)
  sourceId String  locator Json  text String  ordinal Int
  @@index([sourceId, ordinal])
}
model Provenance {
  statementId String  sourceId String  chunkId String
  locator Json  quote String       // verification ONLY. never served (§27).
  extractorVersion String  promptVersion String  modelId String
  extractedAt DateTime
  @@id([statementId, chunkId])
  @@index([sourceId])
}

// ═══════ L2 ENTITY ═══════
model Concept {
  id String @id                    // opaque cuid2. IMMUTABLE. meaningless.
  slug String @unique              // MUTABLE. never an FK target.
  name String
  kind String @default("ENTITY")   // ENTITY | SKILL | ... (registry)
  scope String @default("PUBLIC")  // PUBLIC | tenant:<id>
  status String @default("DRAFT")  // DRAFT|ACTIVE|DEPRECATED|MERGED|SPLIT
  version Int @default(1)  supersedes String?
  mergedInto String?               // tombstone → single target
  splitInto String[]               // tombstone → AMBIGUOUS, resolved by usage
context
  createdAt DateTime @default(now())
  @@index([status, kind])
  @@index([scope, status])
}
model ConceptAlias {
  conceptId String  alias String  language String @default("en")
  kind String @default("SYNONYM")  //
SYNONYM|ABBREV|TRANSLATION|MISSPELLING|FORMER_NAME
  @@id([conceptId, alias, language])
  @@index([alias])
}
model ConceptTag {
  conceptId String  namespace String  value String
  @@id([conceptId, namespace, value])
  @@index([namespace, value])
}
```

```
// ─── THREE graphs. Three tables. Never unified. ───
model DependencyEdge {             // REQUIRES. ACYCLIC (enforced).
  fromId String  toId String  strength Float @default(1)
  contextId String?                // null = cognitive; set = curricular sequenc-
ing
  version Int @default(1)  supersedes String?
  @@id([fromId, toId, version])
  @@index([toId])
}
model CompositionEdge {            // PART_OF. ACYCLIC (enforced). C2.
  partId String  wholeId String  ordinal Int?
  version Int @default(1)
  @@id([partId, wholeId, version])
  @@index([wholeId])
}
model ReinforcementEdge {          // cycles LEGAL and expected.
  fromId String  toId String
  type String   // ENABLES|REINFORCES|REVISITS|TRANSFER_TO|CO_OCCURS
                // |COMMON_CONFUSION|ANALOGOUS_TO|CONTRASTS|SUCCEEDS
  strength Float @default(1)
  earned Boolean @default(false)   // true = derived from observation
  contextId String?
  version Int @default(1)
  @@id([fromId, toId, type, version])
  @@index([toId, type])
}

// ═══════ L3 ASSERTION ═══════
model ContextDimension {           // OPEN REGISTRY. adding one is an INSERT.
  key String @id                   // jurisdiction, physicsRegime, musicTradi-
tion…
  valueType String                 // enum|iso3166|iso639|range|date|string
  values String[]  specificity Int  appliesTo String[]  since String
}
model Context {
  id String @id                    // sha256(canonicalJSON(dimensions)) — see
§13.2
  dimensions Json
  @@index([dimensions], type: Gin)
}
model Statement {
  id String @id
  kind String                      // FACT|PROCEDURE|PRINCIPLE|RULE|… (registry)
  form String                      // SPO|CONDITIONAL|QUANTIFIED|CAUSAL
                                   // |COMPARATIVE|PROBABILISTIC|DEFINITIONAL
  structure Json                   // form-specific. SPO is ONE case.
  subjectId String?                // denormalised SPO index (form=SPO only)
  predicate String?
  objectId String?  objectLit String?
  text String                      // WRITTEN, never copied (§27.2)
  payload Json                     // kind-specific, registry-validated
```

```
  contextId String
  scope String @default("PUBLIC")
  // trust — every field is EVIDENCE, never a conclusion
  trustLevel String @default("MACHINE_PROPOSED")
  validationMethods String[]
  corroborationCount Int @default(0)
  independentSourceCount Int @default(0)
  derivedFrom String[]             // inference chain
  authority String?
  evidenceLevel String?
  version Int @default(1)  supersedes String?
  createdAt DateTime @default(now())
  @@index([subjectId, trustLevel])
  @@index([contextId])
  @@index([predicate, objectId])
}
model Contradiction {
  id String @id  aId String  bId String  form String  contextOverlap Json
  status String                    // OPEN|RESOLVED|COEXIST
  detectedAt DateTime  resolvedBy String?
  @@index([aId])  @@index([bId])
}

// ═══════ L4 TEACHING ═══════
model TeachingAsset {
  id String @id
  kind String                      // MISCONCEPTION|ANALOGY|WORKED_EXAMPLE|…
(registry)
  conceptId String  statementId String?
  payload Json                     // ANALOGY MUST carry breakdownPoint (V14)
  contextId String
  trustLevel String @default("MACHINE_PROPOSED")
  scope String @default("PUBLIC")
  version Int @default(1)  supersedes String?
  @@index([conceptId, kind, trustLevel])
}
model AssetEfficacy {              // DERIVED. never authored.
  assetId String  contextId String
  exposures Int  subsequentSuccess Int  computedAt DateTime
  @@id([assetId, contextId])
}

// ═══════ ASSESSMENT (C3 — in scope, scheduled 2E) ═══════
model AssessmentItem {
  id String @id  kind String       //
MCQ|SHORT|NUMERIC|ORDERING|MATCHING|ARTIFACT|CODE
  prompt String  payload Json      // distractors carry diagnosesMisconception
  contextId String  scope String @default("PUBLIC")
  trustLevel String @default("MACHINE_PROPOSED")
  version Int @default(1)  supersedes String?
}
```

```
model ItemConcept {
  itemId String  conceptId String  role String  bloom String?
  @@id([itemId, conceptId])
}

// ═══════ L5 PROGRAM ═══════
model Program {
  id String @id  slug String @unique  name String
  kind String                       //
SCHOOL_BOARD|DEGREE|CERTIFICATION|EXAM|COURSE|INTERNAL
  authority String  jurisdiction String?
  scope String @default("PUBLIC")  version String
}
model ProgramNode {
  id String @id  programId String  parentId String?
  nodeKind String                   //
DOMAIN|TRACK|LEVEL|MODULE|UNIT|TOPIC|ROTATION|GRADE|PAPER
  name String  ordinal Int  code String?
  @@index([programId, parentId, ordinal])
}
model Mapping {
  programNodeId String  conceptId String
  depth String                      // INTRODUCE|DEVELOP|MASTER|REVISE|ASSUMED
  ordinal Int  examWeight Float?  required Boolean @default(true)
  @@id([programNodeId, conceptId])
  @@index([conceptId])
}
model LearningObjective {
  id String @id  programNodeId String  statement String  bloom String  ordinal
Int
}
model ObjectiveConcept {
  objectiveId String  conceptId String  role String
  @@id([objectiveId, conceptId])
}

// ═══════ GOVERNANCE ═══════
model ReviewEvent {
  id String @id  targetKind String  targetId String
  decision String                   //
APPROVE|REJECT|EDIT|MERGE|SPLIT|DISPUTE|PROMOTE|DEMOTE
  fromTrust String?  toTrust String?
  actorId String                    // a human. always.
  before Json?  after Json?  reason String?  batchId String?
  createdAt DateTime @default(now())
  @@index([targetKind, targetId])
  @@index([actorId])                // reputation
}
model Release       { id String @id  label String  createdAt DateTime  frozen
Boolean }
model ReleaseMember { releaseId String  kind String  entityId String
```

```
                      @@id([releaseId, kind, entityId]) }
model ClosureCache  { conceptId String  releaseId String  closure Json  compute-
dAt DateTime
                      @@id([conceptId, releaseId]) }

// ═══════ L6 OBSERVATION — SEPARATE DATABASE ═══════
model Observation {
  id String @id  learnerId String  taskId String?
  conceptIds String[]
  contextId String                 // REQUIRED. makes future SPLIT resolvable.
  outcome Json  bloomLevel String?
  assetIds String[]  releaseId String
  occurredAt DateTime
  @@index([learnerId, occurredAt])
  @@index([conceptIds], type: Gin)
}
// NO mastery table. Mastery is a query (§20.2).
```

10.1 Deliberately absent

`difficulty` anywhere · a mastery score · `subject` on `Concept` · `chapter` on `Concept` · any curriculum FK on knowledge · a unified `Edge` table · `verified: boolean` · `quality` on assets · any `DELETE` path.

---

11. Knowledge Graph Architecture

11.1 Dependency graph — `REQUIRES`, acyclic 🔒

Why acyclic, argued rather than asserted: if a genuine cycle existed in knowledge dependency, no learner could construct an entry point and the subject would be unlearnable. Every subject is learnable by someone starting from nothing. Therefore dependency has an acyclic core.

Apparent counterexamples all dissolve:

Apparent cycle	Resolution
limits ⇄ derivatives	`derivatives REQUIRES limits`. The reverse arrow is `REINFORCES`.
vocabulary ⇄ reading	reading requires a *minimal* vocabulary — a smaller, different concept
force ⇄ mass	both require a prior operational concept; `F=ma` relates them
grammar ⇄ writing	`REINFORCES` both ways; children write before knowing grammar

Every one is a reinforcement relationship misfiled as a prerequisite. **The DAG constraint's value is that it forces that distinction to be made explicitly.**

11.2 Reinforcement graph — cyclic by design 🔒

`Functions → Derivatives → Optimisation → Functions` is legal and true. So is `Photosynthesis ⇄ Respiration ⇄ Energy`.

`COMMON_CONFUSION` is load-bearing for teaching: learners conflate weight/mass, velocity/acceleration, precision/accuracy. That is a fact about **minds**, not the world, and it tells the Teaching Engine what to disambiguate *before* the conflation forms.

Reinforcement edges should eventually be **earned** from observation — if mastering X measurably improves later performance on Y, that is an edge with evidence. `earned: boolean` distinguishes authored from measured.

11.3 Composition graph — `PART_OF`, acyclic (C2 restoration)

Light reaction PART_OF *photosynthesis*. Neither a prerequisite nor a reinforcement: you do not need the whole to learn the part, and learning the part is not merely reinforcement of the whole. It is **structural containment**, needed for aggregation, topic rollup, and progress summarisation

("you have covered 4 of 7 parts").

Acyclic for the obvious reason: a thing cannot be part of itself transitively.

11.4 Why three tables, not one with a type column

A single table invites a single traversal function, which invites running a topological sort over cyclic edges. Separate tables plus wall rules W6/W7 make that a compile-and-test failure rather than a subtle runtime bug.

11.5 Conflict between graphs 🔒

Binding rule: the same ordered pair may not appear in both `DependencyEdge` and `ReinforcementEdge` in the same direction. If `A REQUIRES B` exists, `A REINFORCES B` is rejected — dependency is the stronger claim and subsumes it. The reverse pair (`B REINFORCES A`) is legal and common, and is exactly the limits/derivatives case.

Validated on write and as a standing whole-graph test.

---

12. Content Engineering Architecture

12.1 Pipeline

```
fetch → parse → clean → normalise → chunk → EXTRACT → validate
     → resolve → corroborate → contradict → promote → review
```

Stages 2–5 are **pure**: identical bytes produce identical chunk ids on any machine at any time. Extraction is the only nondeterministic stage and is quarantined behind `Advice<T>`.

The pipeline **terminates at a trust level, not at a human.** A statement may legitimately stop at `AUTO_VALIDATED` and be teachable to a consumer whose policy accepts that floor.

12.2 Span-preserving cleaning 🔒

```
interface Span { text: string; sourceRange: [number, number]; page: number }
type Doc = Span[]
```

Removing a header drops a span; other offsets stay intact. A quote's character range therefore maps back to the original page after every transformation — which is what makes grounding checkable and highlighted review possible. Doing this on plain strings destroys the mapping irrecoverably.

12.3 Chunking

`chunkId = sha256(sourceId + JSON(locator) + normalisedText)`. Content-addressed, so **re-ingestion is a diff**: an unchanged chapter produces identical ids and zero work; one edited paragraph produces one new id.

12.4 Four extraction passes

Pass	In	Out	Note
1 entities	chunk	concept candidates resolved against graph + aliases	resolution before creation prevents duplicates at proposal time
2 statements	chunk + resolved entities	assertions linking known entities	
3 dependencies	chunk + entities	`REQUIRES` / `PART_OF` / reinforcement, **classified**	**always human-confirmed regardless of trust** — this is what keeps the DAG honest
4 assets	verified chapter	teaching assets	runs only over verified knowledge

12.5 Halt conditions

> 50% batch validation failure → halt the source. >30% duplicate hits → halt, source probably already ingested. Contradiction rate above baseline → halt.

The expensive failure is not a rejected proposal; it is thousands of subtly wrong ones consuming review capacity.

---

13. Teaching Metadata Architecture

13.1 Why this layer is the product

Knowledge: *photosynthesis converts light energy into chemical energy.* True, verifiable, in every textbook, **known by every model.**

Teaching knowledge: *a fifteen-year-old who just learned that plants "make food from sunlight" will assume the plant is eating the sunlight; name that assumption before introducing the equation, or the equation lands on a wrong model and reinforces it.* Not true or false, in no textbook, **not reliably known by any model.**

Grounding makes lessons accurate. Accuracy is commodity. Without L4, a fully grounded lesson can be dull and badly pitched, the quality metric comes back flat, and the wrong conclusion gets drawn.

Binding rule: the Teaching Engine never invents pedagogy. It retrieves and composes. Where assets are absent it falls back to model-generated presentation, clearly marked, and records `teaching.miss`.

13.2 Asset kinds (registry)

Explanation — `INTUITIVE_EXPLANATION`, `FORMAL_EXPLANATION`, `ANALOGY`, `MENTAL_MODEL`, `STORY`, `MEMORY_ANCHOR`. *Demonstration* — `WORKED_EXAMPLE`, `COUNTEREXAMPLE`, `EXPERIMENT`, `SIMULATION`, `REAL_WORLD_APPLICATION`. *Visual* — `DIAGRAM_SPEC` (binds to Agabi's existing 40+ block catalogue), `WHITEBOARD_FLOW`, `ANIMATION_SPEC`. *Misconception* — `MISCONCEPTION`, `MISCONCEPTION_CORRECTION`, `DISCRIMINATION`. *Interaction* — `SOCRATIC_SEQUENCE`, `RETRIEVAL_PROMPT`, `EXERCISE`, `PROJECT`. *Strategy* — `TEACHING_STRATEGY`, `TEACHING_ORDER`, `AGE_ADAPTATION`, `DIFFICULTY_ADAPTATION`, `REVISION_STRATEGY`.

13.3 `ANALOGY.breakdownPoint` is mandatory 🔒

Every analogy is wrong somewhere. Taught without its limit, it **installs a misconception**: the learner extends the mapping past where it holds and is confidently wrong in a way that is hard to detect and harder to unlearn.

"Current is like water in a pipe" is excellent for flow and resistance, and breaks at capacitance, at charge not being consumed, and at signal propagation speed.

Required by schema (V14), not by convention. Impossible to retrofit across thousands of assets.

13.4 Efficacy is observed, never authored

No `quality` column. Efficacy accumulates from L6: did learners who received this asset perform better afterwards? An authored rating is an opinion; a measured one is evidence.

13.5 Phase 2 scope

Build the schema, registry, trust integration, `assetsFor()`, and efficacy structure. Populate **`MISCONCEPTION`**, **`ANALOGY`**, **`WORKED_EXAMPLE`** **only**, for chapters already knowledge-verified. Twenty-plus kinds designed against no source material would be twenty-plus guesses.

---

14. Validation Architecture

#	Gate	Check	Failure
V1	schema	`accept(advice, schema)`	discard batch
V2	payload	registry schema for kind	discard
V3	**grounding**	normalised quote literally contained in chunk	**discard**
V4	quote length	10 ≤ n ≤ 500	discard
V5	originality	`text` not a substring of `quote`	flag — copyright
V6	entity resolution	names resolve or flagged new	flag
V7	**dependency acyclicity**	proposed `REQUIRES` closes no cycle	**reject, show cycle**
V8	composition acyclicity	proposed `PART_OF` closes no cycle	reject
V9	**edge classification**	prerequisite / composition / reinforcement stated	**always human-confirmed**
V10	graph conflict	pair not already in dependency, same direction (§11.5)	reject
V11	context validity	dimensions registered; validFrom < validUntil	discard
V12	units and dimensions	numeric claims dimensionally consistent	flag
V13	scope	tenant proposal references no other tenant	discard + security alert
V14	**analogy breakdown**	every `ANALOGY` states where it fails	**discard**
V15	self-reference	subject ≠ object	discard

Passing every applicable gate **is** `AUTO_VALIDATED` — a defined epistemic state, not a waiting room.

14.1 V3 — grounding

```
const n = (s: string) => s.normalize("NFC")
  .replace(/[‘’]/g,"'").replace(/[“”]/g,'"').replace(/\s+/g," ").trim().toLower-
Case();
if (!n(chunk.text).includes(n(quote))) reject("QUOTE_NOT_IN_SOURCE");
```

Deliberately not fuzzy. A model paraphrasing while claiming to quote is doing exactly what this catches. Returns the char range, which powers highlighted review.

14.2 Context canonical hashing 🔒 (C5)

```
canonical(dims) = JSON.stringify(
  Object.keys(dims).sort().map(k => [k, normaliseValue(registry[k].valueType,
dims[k])])
);
contextId = sha256(canonical(dims));
```

Without deterministic canonicalisation, two identical contexts produce two rows and specificity matching silently fragments. `normaliseValue` is per-dimension-type: ISO codes uppercased, dates to UTC midnight, enums exact-matched.

14.3 Contradiction as a validator 🔒

A new statement conflicting with a higher-trust statement in an overlapping context **cannot promote** above `AUTO_VALIDATED` and is surfaced to review as a *challenge*, not a candidate.

The graph defends itself and improves at it as it grows. Human effort per statement falls with graph size — the only mechanism that makes 100M concepts arithmetically conceivable.

14.4 Per-form contradiction rules (C4)

Form	Conflict rule
`SPO`	same subject + predicate, overlapping context, different object
`DEFINITIONAL`	same definiendum, different definiens — always a conflict
`COMPARATIVE`	same pair + dimension, opposite direction
`QUANTIFIED`	universal vs existential counterexample over the same domain
`CONDITIONAL`	same antecedent, contradictory consequents
`CAUSAL`	same cause + effect, opposite sign
`PROBABILISTIC`	**non-overlapping confidence intervals** on the same quantity

Forms not listed are **explicitly undetectable** and marked as such rather than silently passing — an undetected conflict must never look like an absence of conflict.

---

15. Search Architecture

Search resolves **concepts**, never documents. Output is `ConceptRef[]`.

Rung	Method	Deterministic	Ships
1	exact slug	yes	2A
2	alias (incl. translations, former names)	yes	2A
3	trigram over name, aliases, statement text	yes	2D
4	vector	no	when rung 3 measurably fails

For a syllabus term, exact matching is *better* than semantic — it cannot drift, it is explainable, and it replays identically.

```
interface SearchQuery {
  text?: string; kinds?: string[]; tags?: Tag[]; program?: ProgramNodeId;
  context?: Context;
  relatedTo?: ConceptId;    // reinforcement
  requires?: ConceptId;     // dependency, reverse
  partOf?: ConceptId;       // composition
  trustPolicy: TrustPolicy; // REQUIRED. no default.
  scope?: string;
}
interface SearchHit {
  conceptId: ConceptId; score: number;
  rung: 1|2|3|4; matchedOn: string; trustLevel: TrustLevel;
}
```

`trustPolicy` having no default makes it impossible to accidentally surface machine-proposed knowledge to a student. `rung` and `matchedOn` make every result explainable — a search that cannot say why it matched cannot be trusted in a teaching path.

Coverage: concept, statement, skill, objective, alias, dependency, relationship, tag, curriculum, and future semantic — all resolving to concepts.

---

16. API Architecture

Internal TypeScript modules in Phase 2. No public HTTP knowledge API — one with no consumer cannot be designed correctly.

```
export const knowledge = {
  resolve(q: SearchQuery): SearchHit[];
  getConcept(idOrSlug, at?: ReleaseId): Concept | null;
  statementsFor(ids, ctx, policy: TrustPolicy, at?): Statement[];
  prerequisitesOf(id, depth): OrderedConcept[];   // dependency
  dependentsOf(id): ConceptRef[];                 // dependency reverse
  partsOf(id): ConceptRef[];                      // composition
  reinforcementsOf(id, types): ConceptRef[];      // cyclic
  assetsFor(concepts, ctx, policy): TeachingAsset[];
  itemsFor(concepts, ctx, policy): AssessmentItem[];
  curriculumFor(id): Mapping[];
  conceptsUnder(programNodeId): Mapping[];
  selectPath(seeds, learner, objective, policy, budget): LessonPlan;
};
export const observation = {
  record(o: NewObservation): void;
  mastery(learnerId, conceptId, atBloom?, asOf?): MasteryEstimate;
  efficacy(assetId, ctx): EfficacyEstimate;
};
```

The three graphs are **named apart**, never `getEdges(type)`. `TrustPolicy` is required on every content-returning call.

---

17. Storage Architecture

17.1 Three stores

Store	Holds	Properties
Knowledge (Postgres)	L1–L5	append-only · versioned · public · permanent · low write volume
Observation (Postgres, separate instance)	L6	append-only · high volume · private · erasable · time-partitioned
Derived	closures, search index, efficacy rollups	**rebuildable · never authoritative**

17.2 Why Postgres 🔒

1. **Atomic multi-row writes with auditable history are non-negotiable.** A review commits statements, edges, assets and events together or not at all. Without transactions a partial failure is silent corruption with no consistent state to compare against.
2. **The workload is majority-relational.** Context ranking, trust filtering, text matching and analytics are relational. Genuine graph traversal is bounded (`REQUIRES` closure is single-digit depth by cognitive necessity) and cacheable.
3. Already deployed. Zero new infrastructure for a product with no users.
4. `KnowledgeStore` makes it reversible at the cost of one file.

Rejected: Neo4j/Memgraph (optimises a minority workload; licence lock-in; weaker on the majority patterns) · document stores (no recursive traversal; the model is highly relational) · search indexes as canonical (no transactions, no referential integrity — a projection, never truth) · triple stores (context is second-class; payloads are not triples).

Falsifier: if `REQUIRES`-closure p95 exceeds 50 ms *with* the closure cache, introduce a traversal engine as a derived store behind the same interface. Not before.

---

18. Indexing Strategy

Full index inventory as in the schema, plus:

Partial indexes — most reads touch only high-trust current rows:

```
CREATE INDEX stmt_teachable ON "Statement" ("subjectId","contextId")
  WHERE "trustLevel" IN
('OFFICIAL_SOURCE_VERIFIED','AGABI_CANONICAL','EXPERT_REVIEWED');
```

Closure cache removes dependency traversal from the hot path: `(conceptId, releaseId) → ordered prerequisite list`. **v1 invalidation clears the whole cache on any review commit** — crude, correct, cheap. Subgraph-precise invalidation is deferred (§34-D3), because premature precision risks a staleness bug that is very hard to detect.

GIN trigram on `ConceptAlias.alias` and `Statement.text` for rung 3. **GIN** on `Context.dimensions` and `Observation.conceptIds`.

18A. Identity and IDs 🔒

The most irreversible decision in the platform.

18A.1 The rule

Identity is a **cuid2** — opaque, k-sortable, collision-resistant, **meaningless**. Human readability is a **separate, mutable** `slug` which **no foreign key ever references**. Classification lives in tags.

18A.2 Why meaning must never be encoded

The instinct is readable identifiers: `BIO.PHOTO.CHLORENERGY`. They are pleasant in review screens, URLs and logs. They are also certain to become false, by three independent mechanisms, each of which will occur within a decade:

Failure	Example
Reclassification	`BIO.` asserts Biology. Chlorophyll absorbing light is also Physics. The prefix is now a lie.
Reorganisation	`PHOTO.` asserts a position under photosynthesis. The 2028 NCERT edition reorganises. The id describes a structure that no longer exists.
Multilingual / multi-curricular expansion	A Hindi-medium slug and a CBSE slug for the same concept cannot both be the identity.

Once a meaningful id is referenced by statements, edges, mappings, observations and stored lessons, correcting it means rewriting every reference. There is no migration; there is only living with a database that asserts falsehoods.

Alternatives considered. Readable hierarchical ids — rejected above. UUIDv4 — acceptable but not k-sortable, so index locality is poorer. Natural key on `name` — rejected: names change, and two concepts may share a name across domains. **cuid2 — chosen.**

Consequence. Debugging requires a slug lookup. Mitigated by returning `slug` on every API response. A small permanent cost buying a permanent guarantee.

18A.3 Slugs

`slugify("Chlorophyll") → "chlorophyll"`. Unique, **mutable**. On change the old slug is retained as a `ConceptAlias` with `kind: FORMER_NAME`, so existing links keep resolving. **Never an FK target** — asserted by the `identity` test.

18A.4 Resolution follows tombstones

```
resolveSlug(slug) → ConceptId | AmbiguousSplit | null
  1. current slug
  2. FORMER_NAME alias
  3. follow mergedInto chain (bounded)
  4. if splitInto is set → return AmbiguousSplit, resolved by usage context
(§20.3)
```

A merged concept's id resolves forever, which is what makes merging psychologically safe enough that reviewers will actually do it. A split concept's id resolves **honestly ambiguously** rather than wrongly.

18A.5 ID scheme by entity

Entity	Scheme	Rationale
Concept, Statement, edges, assets, items	cuid2	opaque identity
Context	`sha256(canonicalJSON(dimensions))`	identity **is** content — identical contexts must share a row (§14.2)
SourceChunk	`sha256(sourceId + locator + normalisedText)`	content-addressed → re-ingestion is a diff
Source	`sha256(checksum)`	same bytes = same source
Release	`YYYY-MM-DD-nn`	human-meaningful; releases are immutable by nature

Two deliberate exceptions to opacity. `Context` and `SourceChunk` derive identity from content **because their identity is their content**. That property is what makes §12.3's diff-based re-ingestion possible.

18B. Traversal Engine

18B.1 Bounded by construction 🔒

```
interface TraversalSpec {
  seeds: ConceptId[];
  graph: "dependency" | "composition" | "reinforcement";
  direction: "forward" | "reverse";
  maxDepth: number;        // REQUIRED. no default meaning infinity.
  maxNodes: number;        // REQUIRED. hard stop.
  context?: Context;       // prefer context-matching edges
  at?: ReleaseId;
}
```

`maxDepth` and `maxNodes` are **mandatory with no defaults**. An unbounded traversal over a graph that has accidentally acquired a cycle is a production hang; requiring the bound makes that impossible to write by accident.

Truncation is **reported** (`truncated: true`), never silent — a cap the caller does not know about is indistinguishable from complete coverage.

18B.2 Implementation

```
WITH RECURSIVE closure(id, depth, path) AS (
  SELECT unnest($1::text[]), 0, ARRAY[]::text[]
  UNION ALL
  SELECT e."fromId", c.depth + 1, c.path || e."toId"
  FROM closure c
  JOIN "DependencyEdge" e ON e."toId" = c.id
  WHERE c.depth < $2
    AND NOT e."fromId" = ANY(c.path)      -- cycle guard, belt and braces
)
SELECT DISTINCT id, MIN(depth) AS depth FROM closure GROUP BY id LIMIT $3;
```

The path guard means that even if a cycle somehow exists despite §11.5's three defences, traversal terminates rather than hanging.

18B.3 Topological sort

Kahn's algorithm over the closure subgraph, with **deterministic tie-breaking**: `Mapping.ordinal` when a program is in play, otherwise concept id. Identical graph and seeds always produce identical order — required for `selectPath` to be replayable.

Only the dependency graph is ever sorted. Attempting to sort the reinforcement graph is prevented by W6: `dependency.ts` cannot import it, and the sort lives there.

18C. Skill Model

Skills are **concepts** (`kind: SKILL`), not statements. *"Write a formal letter"* remains the same skill whether or not our rubric for it changes — so identity belongs to the skill, and the rubric, being revisable, lives in attached statements and assets.

```
SkillPayload {
  description: string;
  components: string[];                          // sub-abilities; often concepts
themselves
  rubric: { criterion, weak, adequate, strong, weight }[];
  exemplars: { quality: "weak" | "strong", artifact, commentary }[];
  practiceTasks: { prompt, constraints? }[];
  feedbackDimensions: string[];
}
capabilities: [performable, rubric_scored]
```

18C.1 Why capabilities, not `kind` checks

Consumers must never switch on `kind` — that puts domain knowledge in the Teaching Engine and makes adding a type require changing it. Each type declares **capabilities** (`assertable`, `ordered`, `performable`, `rubric_scored`, `executable`, `time_bound`, `jurisdictional`, `citable`), and consumers ask *"is this performable?"*, never *"is this a skill?"*

When `PERFORMANCE` (music) arrives with the same capabilities, the assessment machinery already handles it with no change.

18C.2 Cross-domain validation

Domain	Skill	Rubric criteria
English	write a formal letter	register, structure, salutation, concision
Law	analyse a source	authority identified, ratio extracted, distinguished
Music	vibrato	pitch centre, width, evenness, musical appropriateness
Programming	debug systematically	hypothesis formed, isolated, verified, minimal fix
Medicine	take a history	completeness, sequence, empathy, red flags

One payload shape, five domains. The shape holds.

18D. Learning Objectives and Time Estimates

18D.1 Objective assignment — pipeline stage 5

Objectives are **program artifacts, not knowledge** (§3.1 Q7 versus Q2). CBSE and IB may share concepts and want different objectives from them, so an objective is never a property of a concept.

Extraction pass 5 runs over a **curriculum document**, not a textbook:

```
curriculum doc → parse → extract objective statements
               → classify Bloom level
               → link to concepts as PRIMARY | SUPPORTING | ASSUMED
               → human review (same ladder, same door)
```

An objective is *satisfied* when its `PRIMARY` concepts are mastered at its Bloom level. That evaluation belongs to Phase 3; the linkage ships now so the data exists when the Mastery Engine arrives.

18D.2 Estimated learning and mastery time 🔬

Listed as concept properties in the brief. **Never authored** — they are conclusions, and P4 forbids storing conclusions.

Both are **derived from observations**:

Estimate	Derivation
estimated learning time	median elapsed time between first exposure and first successful application, per learner cohort
estimated mastery time	median elapsed time to sustained success across spaced retrievals

Until L6 has data, both return `INSUFFICIENT_DATA` — never a fabricated number. Same honesty rule as `NOT_INSTALLED` health providers: an invented estimate is worse than an absent one, because a planner will act on it.

Intrinsic proxies available immediately from the graph, and all computed rather than authored: prerequisite depth, prerequisite count, dependent count, statement count, and the authored `bloom` tag — the one exception, because cognitive operation is a property of the knowledge rather than of the learner.

Part III — operations, trust, security, assurance.

Part III — Operations, Trust, Assurance

19. Versioning Strategy 🔒

Nothing is overwritten. Editing creates a row; the old becomes `DEPRECATED` and stays readable forever.

Why not mutate-plus-audit-log: reconstructing old state means replaying the log backwards — fragile and slow. *Why not temporal tables:* Postgres-specific, violating engine independence.

Entity	Versioned	Note
Concept	yes	rare — identity is stable by design
Statement	**yes**	the common case
Dependency / Composition / Reinforcement edge	yes	reinforcement increasingly *earned*
TeachingAsset	**yes**	assets improve; old ones must replay
AssessmentItem	yes	
Mapping	yes	syllabi change between editions
ConceptTag / ConceptAlias	**no**	additive; history in `ReviewEvent`
Context	**no**	immutable by construction — id is its hash
Observation	**no**	an append-only fact about a moment

Trust level is versioned with the row. *"Was this `OFFICIAL_SOURCE_VERIFIED` when we taught it in March?"* must be answerable; a mutable trust column would make every historical trust claim unverifiable.

Releases pin exact version ids. Every lesson records its release, so a 2029 replay of a 2026 lesson resolves every statement and asset to the version the learner actually saw.

---

20. Knowledge Evolution Strategy

20.1 Five kinds of change

Change	Mechanism	History
Correction — it was wrong	new version, old `DEPRECATED`	full
Refinement — it was imprecise	new version	full
Supersession — reality moved	new statement; old gets `validUntil`	**both current**
Retraction — affirmatively false	`RETRACTED`, no replacement	retained, marked
Reclassification	tag update	`ReviewEvent`

Supersession is **not** correction. The 2018 hypertension guideline was not wrong in 2018. Deprecating it falsifies history; a `validUntil` records the truth.

20.2 Mastery is a query 🔒

```
mastery(learnerId, conceptId, atBloom?, asOf?) → MasteryEstimate
```

Computed from observations by whatever model is current. Improving from naive percentage to Bayesian knowledge tracing **reinterprets all history** instead of invalidating it. A stored mastery row could not have been recomputed.

Representable as a result: mastery at recall but not application, decay over time, transfer failure, misconception-specific failure, confidence versus accuracy. None of these fit `(userId, conceptId)`.

20.3 Merge and split

Merge — move aliases, tags, statements, all three edge kinds, assets and mappings to the winner; tombstone the loser with `mergedInto`; the loser's id resolves forever. Non-destructive, which is what makes reviewers willing to do it.

Split 🔒 — the direction reality forces. A reviewer discovers `c_energy` has meant *kinetic energy* in Physics contexts and *energy generally* in Biology contexts.

```
split(sourceId, [targetA, targetB], plan):
  1. create targets
  2. re-attribute each statement       (per item, or by context rule)
  3. re-attribute each edge            (all three graphs)
  4. apportion observations            BY THE CONTEXT ON EACH OBSERVATION
  5. tombstone source with splitInto[] → resolution becomes AMBIGUOUS
  6. ReviewEvent records the full plan
```

Step 5 is honest rather than convenient: after a split, an old reference resolves to *two* things, and any reference that did not record its usage context is genuinely ambiguous. **This is why every concept reference carries `contextId` —** one column now, unrecoverable later.

20.4 Cascading review

A changed statement flags: assessment items evidencing its concept, statements citing it, teaching assets teaching it, mappings at `MASTER` depth. Flagged means queued — a human decides. **Lessons already taught are never retroactively changed.**

20.5 Source deprecation

A withdrawn source demotes statements sourced *only* from it to `DISPUTED`. Statements with independent corroborating provenance are unaffected — the concrete payoff of allowing multiple provenance rows.

---

21. Migration Strategy

21.1 Into the platform

Nothing to migrate; Agabi stores no knowledge. Phase 2 is additive: new tables, a second database, one changed call site.

Rollback stays trivial through 2A–2D: revert `startLesson` to `defaultOutline(topic)`. Knowledge tables go inert. No data loss.

21.2 Migrations the architecture prevents

Would-be migration	Prevented by
add a context dimension	open registry
add a knowledge / asset / item type	registries
add a trust level	ladder is data
separate prerequisite from reinforcement	three tables already
add pedagogy	L4 exists
recompute mastery under a better model	mastery is a query
split a conflated concept	first-class; references carry context
add a curriculum	mapping layer
add a tenant	`scope` from row one
erase a learner without touching knowledge	separate stores
recover overwritten knowledge	never overwritten
backfill provenance	mandatory from row one
add difficulty semantics	never stored
rename after reclassification	opaque ids

Fourteen migrations prevented by decisions that cost nothing today. **This table is the phase's actual deliverable** — it is what "no redesign required" means concretely.

21.3 Migrations that will legitimately happen

Authored reinforcement edges → earned. Trigram → vector. Misconception payload → first-class concept. Observation partitioning. Traversal engine as a derived store. All additive; none re-authors content.

---

22. Scaling Strategy

Concepts	Statements	Assets	Observations	Architecture
10^2	10^3	10^2	10^3	single instance
10^4	10^5	10^4	10^6	+ closure cache
10^6	10^7	10^6	10^9	+ read replica, search index, observation partitioning
10^8	10^9	10^8	10^{12}	+ partitioning, + traversal engine if measured

Observations dominate by three orders of magnitude — which is why they are a separate store with independent partitioning and retention.

Nothing in the logical model changes at any step. Every change is additive and operational.

The real constraint is not technical. 100M concepts is 10,000 person-years of verification. Reaching it is a *contribution* problem: federated review, reputation weighting, and accepting bulk knowledge at lower trust tiers. The architecture supplies the primitives (`ReviewEvent.actorId`, trust levels, `scope`); the workflow is out of scope and §34 records it as deferred.

---

23. Multi-Tenancy Strategy

`scope` on every knowledge row: `PUBLIC` or `tenant:<id>`.

Rule	Enforcement
tenant knowledge is visible only within its tenant	filter applied **inside** **`KnowledgeStore`**, never by callers
tenant knowledge MAY reference public concepts	one-way; validated by V13
public knowledge MUST NOT reference tenant knowledge	V13; a violation is a security alert
a tenant may not read another tenant	store-level; conformance-tested
observations are tenant-scoped	separate store, same rule

Enforcement lives in the store because caller-side filtering is one forgotten `where` clause away from a leak. The conformance suite includes a two-tenant fixture asserting zero cross-visibility across every read method.

Phase 2 ships the field and the enforcement. Tenant administration, billing and onboarding are out of scope.

---

24. Research Connector Architecture

```
export interface SourceConnector {
  id: string;
  kinds: SourceKind[];
  license(ref: string): Promise<LicenseInfo>;   // called BEFORE fetch
  fetch(ref: string): Promise<{ source: RawSource; bytes: Buffer }>;
  rateLimit?: RateLimitPolicy;
}
```

`license()` runs first and can **refuse ingestion**. Unknown licence requires explicit human approval before fetch.

Connector	Kind	Licence posture	Phase
local filesystem (PDF/MD/HTML)	manual	operator asserts	2A
NCERT PDFs	book	⚠️ free download ≠ free reproduction	2A
Government curriculum	dataset	usually permissive	2D
Wikipedia / Wikidata	web	CC-BY-SA — attribution obligations	later
OpenStax	book	CC-BY	later
arXiv / PubMed	paper	mixed	later
Web crawler	web	⚠️ per-domain assessment	later

Binding invariant: research never writes to the graph. Every connector produces `MACHINE_PROPOSED` knowledge into the identical pipeline. There is no privileged path and no trusted publisher — a connector cannot weaken the trust guarantee because promotion is a separate, gated process.

---

25. Review Pipeline

25.1 Throughput is the design target

Approach	Rate
individual cards, no source	~60/hr
batched, source in view, quote highlighted	~200/hr
+ auto-reject filtering	~250/hr
+ leverage ordering (usable at 40% coverage)	effective 2×

The gap between row 1 and row 4 is the gap between a project that finishes and one that does not.

25.2 The batch screen

Source passage on the left with the quote highlighted in place; ~8 proposals on the right; approve-all is one keystroke; rejection is per item.

Three properties make it fast: the passage is read once for eight decisions; verification is by *comparison* rather than recall (possible only because of span-preserving cleaning and the char range from V3); and the common case — extraction is mostly right — is one action.

25.3 Queue ordering

```
priority = w₁·dependentCount      // leverage: errors here propagate
         + w₂·knowledgeMissCount  // real student demand
         + w₃·programWeight       // exam importance
         − w₄·ageInQueue          // starvation guard
```

`knowledgeMissCount` comes from the evidence log. **Review follows observed demand, not curriculum order** — which is what makes the first 200 concepts disproportionately valuable.

25.4 Never asked of a reviewer

Assign difficulty (does not exist) · assign an id (machine-minted) · write from scratch (they edit) · check quote presence (machine) · check payload shape (machine) · find duplicates (machine-surfaced) · judge more than ~8 items per screen.

Human attention is spent only on judgment: is it true, is it well-stated, is it the right shape, is the wording original, and — for V9 — is this a prerequisite or a reinforcement.

---

26. Trust Model 🔒

26.1 The ladder

Level	Established by	Human effort	Teachable to
`MACHINE_PROPOSED`	extraction	none	internal R&D only, labelled
`AUTO_VALIDATED`	all applicable validators pass	none	research exploration, labelled
`COMMUNITY_REVIEWED`	≥N reputation-weighted reviewers	distributed	general learning, labelled
`EXPERT_REVIEWED`	credentialed domain reviewer	expert	professional domains
`OFFICIAL_SOURCE_VERIFIED`	matches a designated authority	verification	**exam preparation**
`AGABI_CANONICAL`	editorial decision, typically resolving a conflict	highest	anything

26.2 Promotion is a deterministic function

```
promote(s) =
  AGABI_CANONICAL           if editorial ReviewEvent
  OFFICIAL_SOURCE_VERIFIED  if ReviewEvent by verifier AND authority ∈ designated
  EXPERT_REVIEWED           if ReviewEvent by credentialed expert
  COMMUNITY_REVIEWED        if Σ(reviewer reputation) ≥ threshold
  AUTO_VALIDATED            if all applicable validators pass
                            AND independentSourceCount ≥ 1
                            AND no OPEN contradiction with a higher-trust state-
ment
  MACHINE_PROPOSED          otherwise
```

Anything above `AUTO_VALIDATED` requires a `ReviewEvent` with a human `actorId`. No exception, no threshold, no override. Enforced by test.

26.3 Demotion is automatic and immediate

A raised contradiction, a retracted source, an opened dispute, or a newly-failing validator demotes within one job cycle. **Trust is not a ratchet.** The failure mode being prevented is that nobody gets around to it.

26.4 Admission is declared by the consumer

```
interface TrustPolicy { minimum: TrustLevel; labelBelow: TrustLevel; refuseBelow:
TrustLevel }
```

Exam prep: `OFFICIAL_SOURCE_VERIFIED`. Clinical: `EXPERT_REVIEWED` + authority. General school: `COMMUNITY_REVIEWED`. Research: `AUTO_VALIDATED`, labelled. R&D: `MACHINE_PROPOSED`, labelled.

The invariant no policy may relax:

> The platform never silently presents uncertain knowledge as fact.

26.5 Scaling trust without scaling work

Deterministic validators (zero cost) · cross-source agreement by *publisher* independence, not document · **contradiction detection against verified knowledge, which makes effort per statement fall as the graph grows** · inference with recorded derivation, inheriting the weakest premise's level · reputation-weighted community review.

---

27. Security Model

Threat	Mitigation
Prompt injection in a source document	the extractor's return type **has no trust field** — a model cannot express verification. Structurally impossible, not filtered.
Knowledge poisoning	human review above `AUTO_VALIDATED`; extraction output is data, never instructions
Unauthorised promotion	role check + `actorId` on every `ReviewEvent`
Tenant leakage	store-level filter, conformance-tested (§23)
Source spoofing	`Source.checksum`; connector fetch metadata
Destructive write	no delete methods exist on `KnowledgeStore`
Review flooding	halt conditions; per-source rate limits

Roles: `reader` (trusted public knowledge) · `reviewer` (see `PROPOSED`; approve/reject/edit/merge/split) · `ingestor` (run pipelines) · `admin` (releases, source deprecation, vocabulary). **No role writes above `AUTO_VALIDATED` outside `applyReview`.** `admin` is not a superuser over truth.

27.1 Copyright — a launch gate

Facts are not protectable; expression is. NCERT is free to download, not to reproduce.

`statement.text` is **written, not copied** (V5 flags substring-of-quote). The verbatim quote lives only in `Provenance`, is used only for verification, and is **never served** — conformance-tested. `Source.license` recorded per document; connectors refuse incompatible licences.

27.2 Privacy — a launch gate

Knowledge contains no personal data; observations do. They are **separate stores**, so a DPDP erasure request cannot damage the knowledge graph. India's DPDP Act requires verifiable guardian consent for under-18s — unresolved, and gating public launch alongside copyright.

---

28. Observability Integration

Phase 2 consumes the existing evidence spine rather than duplicating it.

Concern	Owner
what happened during a lesson	evidence spine
which concepts/versions/release a lesson used	evidence spine (`lesson.grounded`)
which extractor and prompt produced a proposal	knowledge provenance — **permanent**
who promoted a statement and when	`ReviewEvent` — **permanent**
topic had no knowledge / no assets	`knowledge.miss` / `teaching.miss`

Rule: operational history may age out; knowledge provenance may never.

Health providers registered with the existing framework, each returning real status or `NOT_INSTALLED` — never a green light for something absent:

Provider	`UP`	`DEGRADED`	`UNSAFE`
knowledge-store	reachable, p95 in budget	latency high	unreachable
knowledge-integrity	0 cycles, 0 missing provenance	duplicate rate above threshold	**cycle present or provenance missing**
trust-pipeline	promotions moving	backlog growing	promotion above `AUTO_VALIDATED` without a `ReviewEvent`
ingestion	healthy	backlog growing	halted
observation-store	reachable	lag	unreachable

`knowledge-integrity` at `UNSAFE` means the graph asserts things it cannot justify. **Teaching degrades to ungrounded rather than serving unjustifiable knowledge.**

Metrics, each with a decision attached: concepts verified/hour · queue depth and age · duplicate rate · grounding rejection rate · golden-set precision/recall · `knowledge.miss` by topic · grounded vs asset-supported vs ungrounded lesson quality · dependency-closure p95 · cycle

count (must be 0) · statements without provenance (must be 0) · promotions lacking a `ReviewEvent` (must be 0).

---

29. Testing Strategy

Standing invariants over data, not unit tests of code:

Test	Asserts	Guards
`dependency-dag`	`REQUIRES` acyclic	S12
`composition-dag`	`PART_OF` acyclic	C2
`reinforcement-cycles-pass`	cycles there **do not fail**	premortem 5
`graph-conflict`	no pair in both dependency and reinforcement, same direction	§11.5
`grounding`	quote literally in source for everything above `MACHINE_PROPOSED`	S4
`provenance`	every statement above `MACHINE_PROPOSED` has provenance	S4
`trust-gate`	no promotion above `AUTO_VALIDATED` without a human `ReviewEvent`	26.2
`trust-demotion`	contradiction demotes within one cycle	26.3
`no-silent-uncertainty`	nothing below `labelBelow` returned unlabelled	S3
`analogy-breakdown`	every `ANALOGY` has a breakdown point	13.3
`split-resolvable`	every concept reference records usage context	20.3
`identity`	no FK targets `slug`; `Concept.id` never updated	S11
`no-delete`	no destructive path	S5
`store-separation`	no query joins knowledge and observation	17.1
`tenant-isolation`	two-tenant fixture, zero cross-visibility	S15
`curriculum-independence`	drop all programs, graph still teachable	S1
`empty-platform`	full suite green on zero rows	S10
`context-canonical`	identical dimension sets hash identically	C5
`conformance`	any store implementation passes	S9

Test	Asserts	Guards
`quote-never-served`	no student-facing response contains `Provenance.quote`	27.1

Determinism: identical bytes → identical chunk ids; fixed graph → identical closure order; table-driven context matching; snapshot path selection.

Golden set: one chapter hand-authored as ground truth. Every extractor, prompt, chunk-size or model change is scored on precision, recall, grounding rate, duplicate rate. Without it, *"the extractor improved"* is an opinion.

---

30. Risk Analysis

#	Risk	Severity	Detection	Mitigation
R1	**content never populated**	terminal	coverage flat	graceful degradation; miss-driven priority; 2A is one chapter end to end
R2	trust ladder gamed by volume	high	promotion vs review rate	contradiction gate; publisher-level independence; halt conditions
R3	**silent conflation**	high, **silent**	mastery anomalies, late	split first-class; usage context on every reference; scheduled conflation report
R4	silent duplication	high, silent	similarity report	aliases; dedupe stage; merge queue; standing metric
R5	prerequisite/reinforcement misclassified	high	learners blocked or mis-sequenced	V9 always human-confirmed
R6	grounded-looking nonsense	high	golden set	V3 machine-checked, never model-judged
R7	bad teaching asset installs a misconception	high	efficacy, late	mandatory breakdown point; trust ladder applies to assets
R8	copyright	terminal	–	written not copied; quotes never served; licence per source; legal gate
R9	DPDP / minors	terminal	–	separate store; `purgeUser`; legal gate
R10	uncertain shown as fact	severe	`no-silent-uncertainty`	labelling invariant, tested
R11	DAG check overreaches	medium	reinforcement cycles fail CI	dedicated pass-test
R12	grounding does not help	strategic	2B vs 2D	falsifiable prediction; stop if 2D also flat
R13	tenant leakage	terminal	conformance	store-level filter

R3 is the one to fear: silent, compounding, visible only years later as inexplicable mastery behaviour.

---

31. Red Team Analysis

Ten claims attacked. Four died, two were reinstated after the attack itself proved wrong, one wounded, one demoted, one corrected, one intact. Four of the six original deaths had been marked LOAD-BEARING — the confidence markers were miscalibrated exactly where it costs most.

Attack	Outcome
”You will never fill it”	**stands** — R1, mitigated by degradation and demand-driven review
”Extractor produces confident nonsense”	mitigated — V3 machine-checked, golden set, revisable trust
”Global concepts fill with duplicates”	mitigated — aliases, dedupe, merges, standing metric
”Identity encodes meaning and lies”	**prevented** — opaque ids, mutable slugs, tags
”Context is over-engineering”	**rejected** — the school case uses one row of nulls; retrofitting is an unanswerable question about every historical row
”Postgres collapses at 100M”	mitigated — shallow traversal, closure cache, store interface
”Nobody uses it, teaching is fine”	**partly landed** — produced L4; grounding alone was never the differentiator
”You are storing copyrighted text”	mitigated — §27.1, legal gate
”Prerequisites aren’t acyclic”	**defeated** — knowledge dependency vs learning process; three graphs
”Binary gate is arithmetically impossible”	**landed** — trust ladder
”Mastery on (user, concept) stores a conclusion”	**landed** — observations + query
”Statements-as-SPO can’t express most knowledge”	**landed** — seven forms, SPO is one index

32. Premortem

1 · The platform was finished and the product never was. 600 concepts; teaching still `defaultOutline` for 95% of topics. → 2A is one chapter, student-visible; every milestone asks "can a student see this?"

2 · The trust ladder became a loophole. Under coverage pressure the exam floor was quietly lowered. Nothing broke visibly; students learned wrong things and blamed themselves. → trust policy is code, reviewed and tested; lowering a floor is a diff; `no-silent-uncertainty` fails CI.

3 · Silent conflation. `c_energy` meant two things for years; mastery transferred where it shouldn't. → split designed; usage context recorded; scheduled report for concepts whose statements cluster into disjoint context groups.

4 · The teaching layer stayed empty. L4 shipped as schema; coverage always looked more urgent; lessons were accurate and forgettable; grounding showed no benefit and was read as "the platform doesn't help." → 2D precedes the expensive breadth work, and the prediction in §33.1 is stated in advance so the result is not misread.

5 · The graphs were re-unified "for simplicity". The DAG check began failing on legitimate reinforcement cycles, so it was disabled; six months later prerequisites contained cycles and path planning silently produced nonsense. → W6/W7 import rules plus a test asserting reinforcement cycles pass.

6 · Legal. Verbatim text in a public database; minors' observations without consent. → §27, both gating launch.

7 · Never proven to help. Nobody measured; allocation became opinion; the graph lost to features with visible metrics. → grounded / asset-supported / ungrounded recorded per lesson; the comparison is a query.

---

33. Future Extension Strategy

Future need	Extension	Cost
new domain (medicine, law, music)	register context dimensions + knowledge kinds	inserts
new curriculum / board / exam	Program + ProgramNode + Mapping rows	inserts
new relationship semantics	reinforcement `type` value	insert
new teaching modality	asset kind	insert
new assessment format	item kind	insert
semantic search	rung 4 behind `resolve()`	one module
graph engine	second `KnowledgeStore`	one module
federated contribution	reputation over existing `ReviewEvent.actorId`	new service, no schema change
AI-generated knowledge at scale	enters as `MACHINE_PROPOSED`; ladder unchanged	none
multimodal sources	new `SourceConnector` + parser	one module

Nothing in this table requires a schema migration to a core table. That is the operational definition of “no redesign required.”

33.1 The falsifiable prediction

Grounding alone (2B) produces a **small** accuracy gain and **no** perceived-quality gain. The large gain arrives with 2D, when misconceptions and analogies enter. Stated in advance so it can be wrong. If 2D is also flat, the thesis is wrong and the plan must change.

---

34. Deferred Decisions

#	Decision	Deferred because	Resolved by	Default until then
D1	single-pass vs four-pass extraction	needs golden-set measurement	2A	four-pass
D2	dedupe similarity threshold	needs observed false/missed merge rates	2C	0.85, never auto-merge
D3	closure cache invalidation granularity	needs observed edge-write patterns	2D	clear all on review commit
D4	community review quorum N	needs reviewer reputation data	post-2C	expert-only
D5	assessment item calibration model	needs response data	Phase 3	store evidence only
D6	which teaching asset kinds beyond three	needs efficacy data	post-2D	three kinds
D7	reinforcement edge strength derivation	needs observation volume	Phase 3	authored, `earned=false`
D8	federated contribution workflow	needs contributors > 1	when true	single reviewer
D9	vector search adoption	needs rung-3 failure evidence	when measured	rungs 1–3
D10	traversal engine adoption	needs p95 > 50 ms with cache	when measured	Postgres

Every deferred decision has a default, a trigger and a resolver. None blocks implementation, and none is load-bearing — which is the correction to RFC-1's most dangerous property.

---

35. Final Architecture Review

The criteria this document must pass to be frozen. Each is checkable, not rhetorical.

#	Criterion	Status
A1	Every deliverable present	✅ 35/35
A2	No contradiction between sections	✅ seven found in inputs, all resolved in §0
A3	Every entity appears once, named consistently	✅ C1 resolved — `Statement` throughout
A4	Every load-bearing claim argued, not asserted	✅ acyclicity, identity, context, trust, split
A5	Every guess marked and given a resolution trigger	✅ §34
A6	Every principle enforced by a named test	✅ §29
A7	No domain-specific assumption in a core table	✅ verified against nine domains
A8	Every schema field justified	✅ §10 + absent-list
A9	Rollback exists for every phase	✅ §21.1
A10	Falsifiable	✅ §33.1
A11	Implementable without further architectural questions	✅ blueprint

35.1 Known limitations, stated honestly

1. **Teaching asset sourcing is unsolved at scale.** Pedagogy lives in teachers and research, not textbooks. The architecture holds it; where it comes from beyond the first three kinds is D6.
2. **Community review has never run.** Reputation weighting is designed against zero reviewers.
3. **Split has never been executed.** Designed carefully; the first real one will teach us something.
4. **10,000 person-years remains 10,000 person-years.** The ladder and the self-defending graph reduce the constant, not the order.

Stating these is the point of A5. An architecture that claims no limitations is concealing them.

35.2 Freeze

On approval this document is **frozen**. Changes require a written amendment recording: what changed, which section, which failure prompted it, and which test now guards it.

Everything after this is implementation.

End of the Final Architecture Baseline.